

LangGraph Multi-Agent Systems



SoftUni Team
Technical Trainers



SoftUni



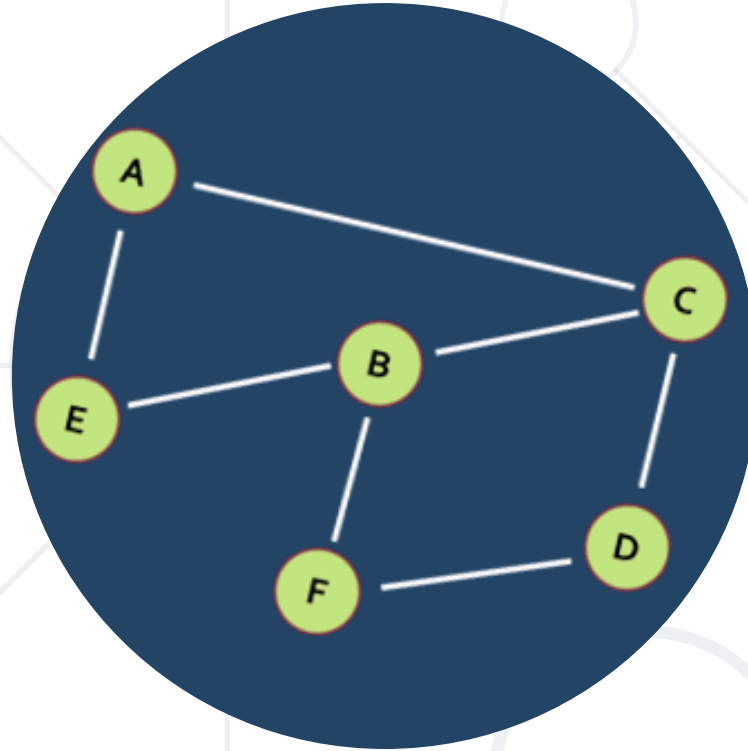
Software University

<https://about.softuni.bg>

Table of Contents

1. What is a Graph?
2. LangGraph Introduction
3. LangGraph Capabilities
4. Multi-Agent Systems

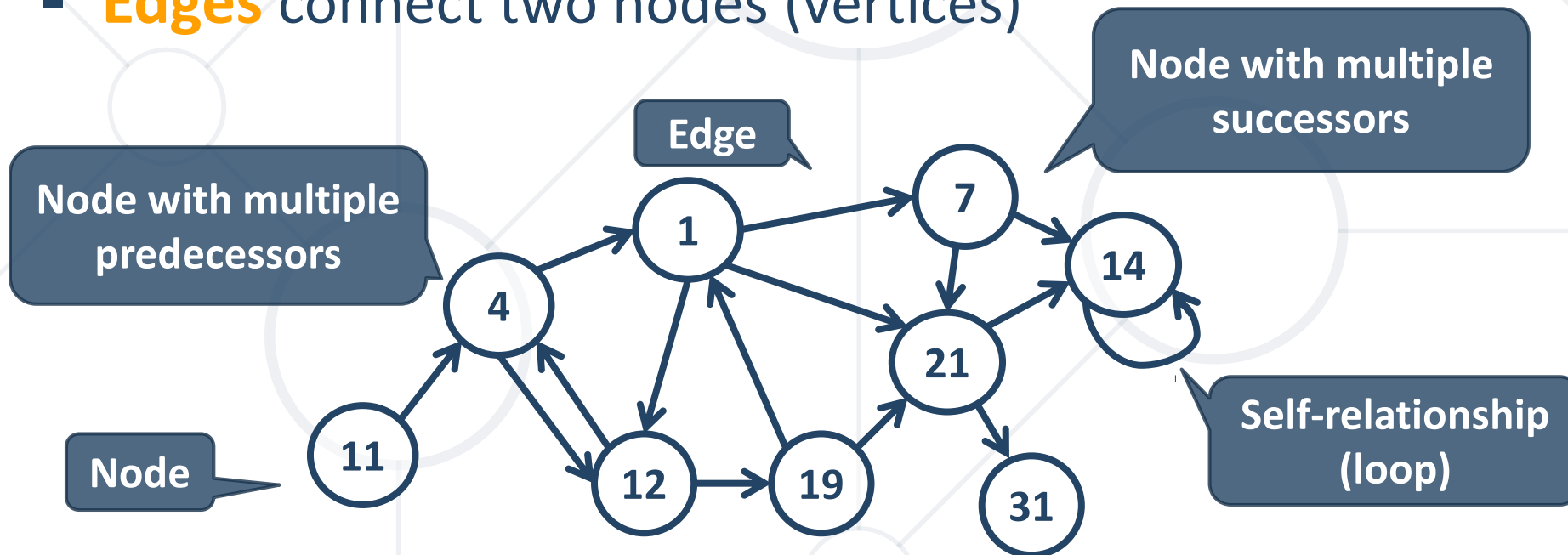




Graphs

Definitions and Terminology

- **Graph**, denoted as $G(V, E)$
 - Set of nodes **V** with many-to-many relationship between them (edges **E**)
 - Each **node (vertex)** has **multiple** predecessors and **multiple** successors
 - **Edges** connect two nodes (vertices)



Graph Definitions (1)

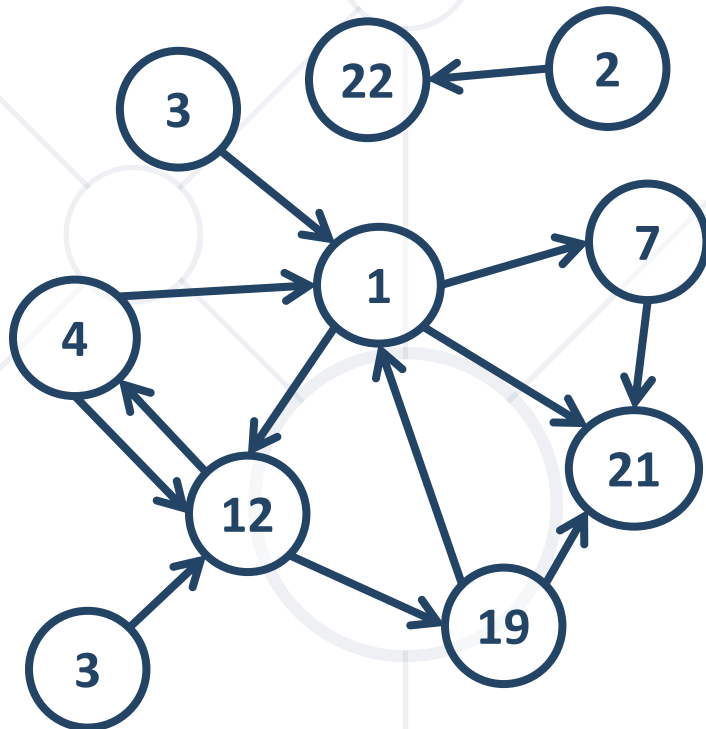
- **Node** (vertex)
 - Element of a graph
 - Can have name / value
 - Keeps a list of adjacent nodes
- **Edge**
 - Connection between two nodes
 - Can be directed / undirected
 - Can be weighted / unweighted



Graph Definitions (2)

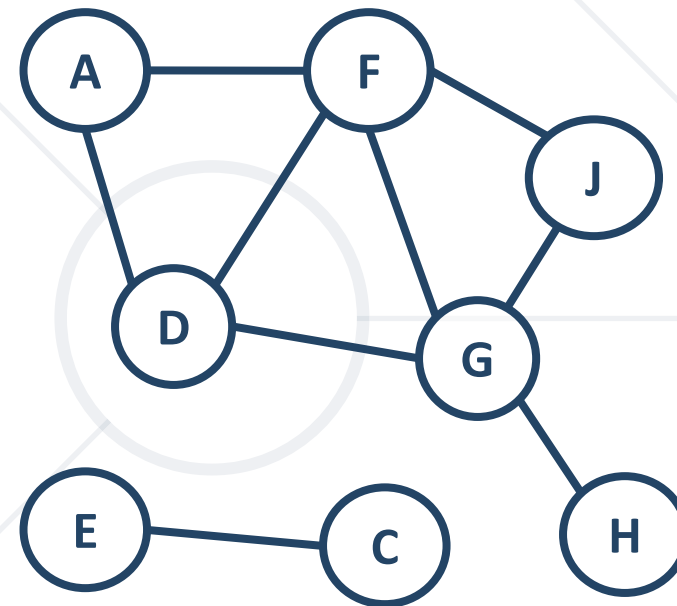
- **Directed graph**

- Edges have direction

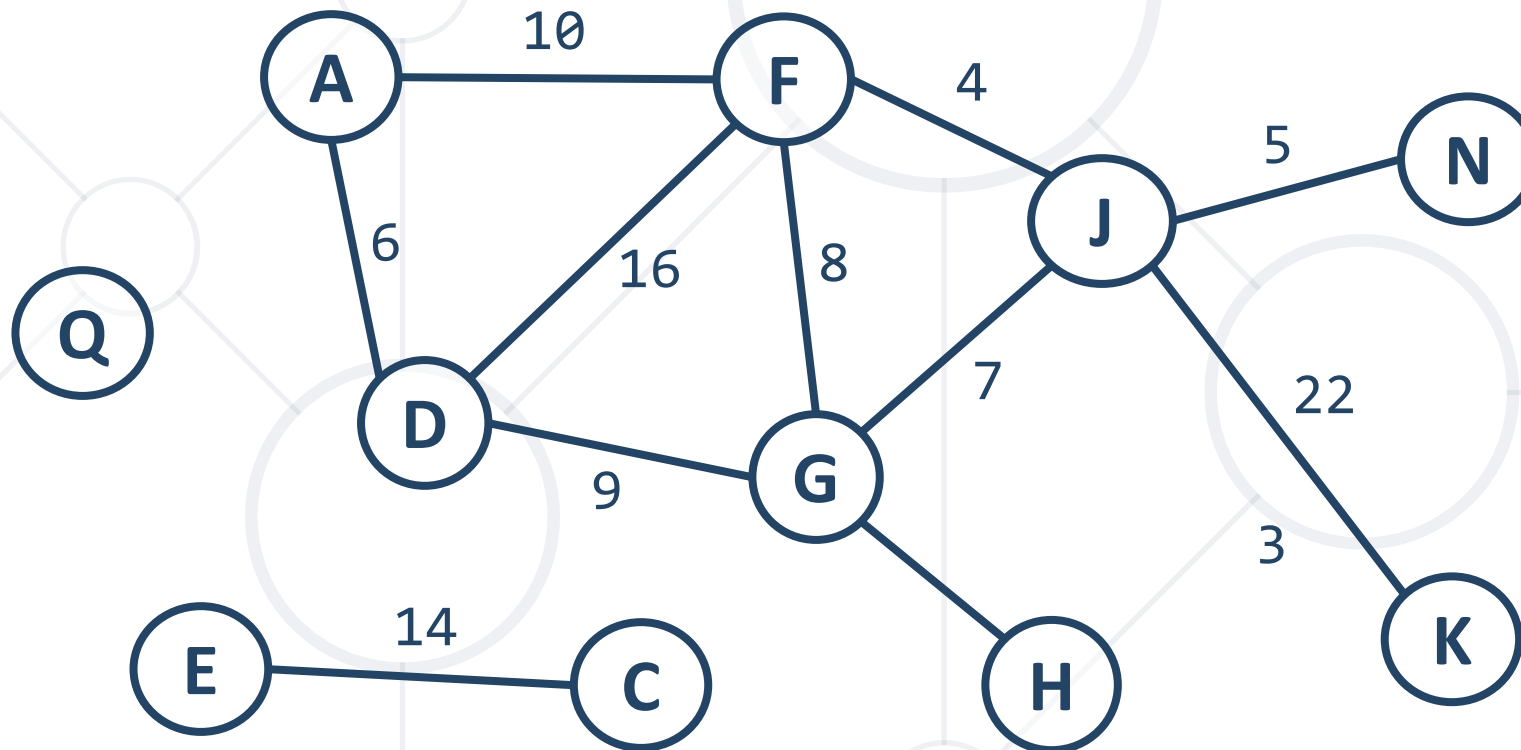


- **Undirected graph**

- Undirected edges



- **Weighted graph**
 - Weight (cost) is associated with each edge

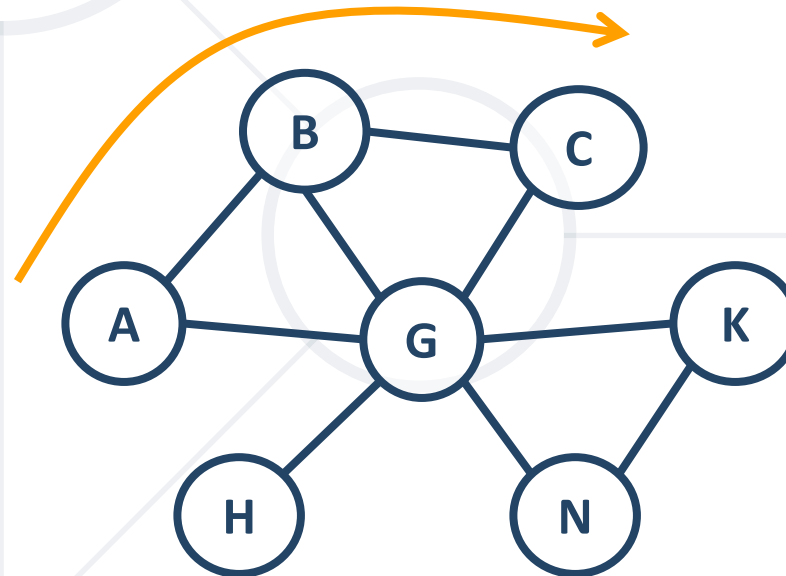


- **Path** (in undirected graph)

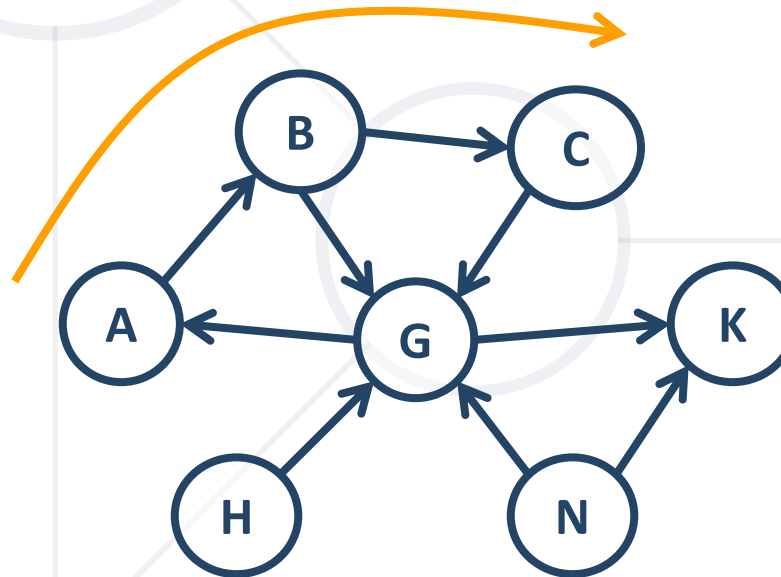
- Sequence of nodes $n_1, n_2, \dots, n_k$
- Edge exists between each pair of nodes n_i, n_{i+1}

- Examples:

- A, B, C is a path
- A, B, G, N, K is a path
- H, K, C is not a path
- H, G, B, G, N is a path



- **Path** (in directed graph)
 - Sequence of nodes $n_1, n_2, \dots, n_k$
 - Directed edge exists between each pair of nodes n_i, n_{i+1}
 - Examples:
 - A, B, C is a path
 - N, G, A, B, C is a path
 - A, G, K is not a path
 - H, G, K, N is not a path



- **Cycle**

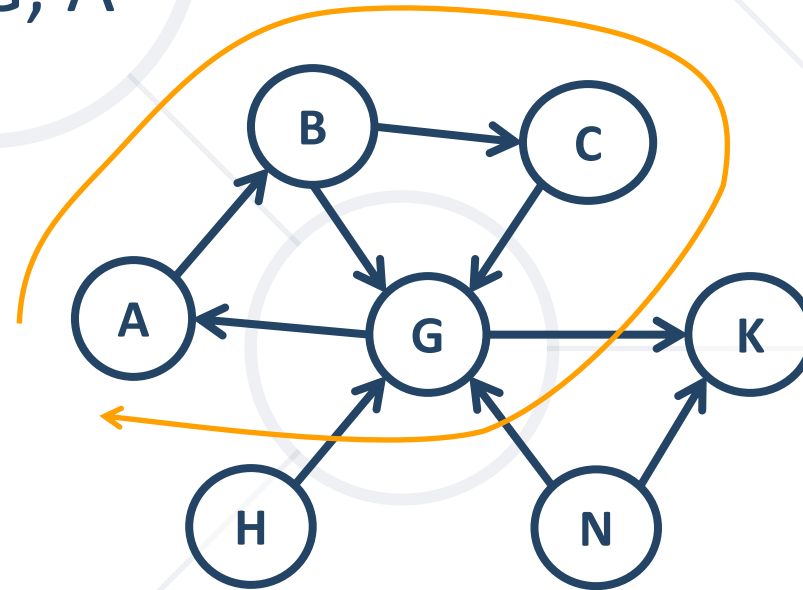
- Path that ends back at the starting node
- Example of cycle: A, B, C, G, A

- **Simple path**

- No cycles in path

- **Acyclic graph**

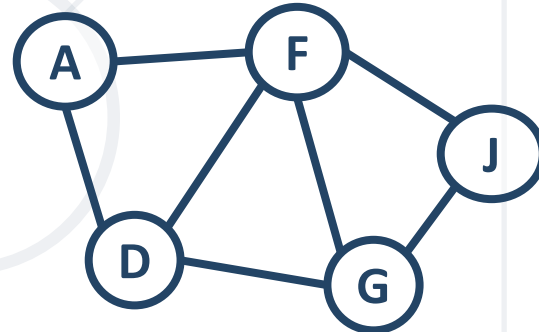
- Graph with no cycles
- Acyclic undirected graphs are trees



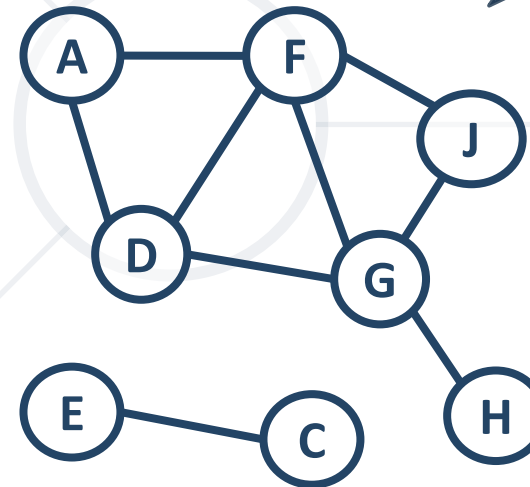
Graph Definitions (7)

- Two nodes are **reachable** if a path exists between them
- **Connected graph**
 - Every two nodes are reachable from each other

Connected
graph

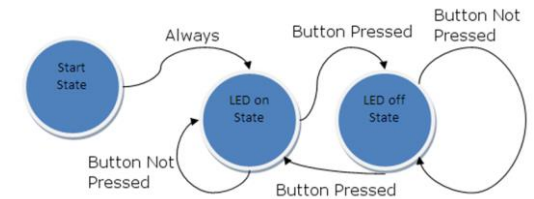


Unconnected graph
holding two connected
components



Graphs and Their Applications

- Graphs have many real-world applications
 - Modeling a **computer network** like the Internet
 - Routes are simple paths in the network
 - Modeling a city **map**
 - Streets are edges, crossings are vertices
 - **Social networks**
 - People are nodes and their connections are edges
 - **State machines**
 - States are nodes, transitions are edges





LangGraph Introduction

Moving from linear chains to cyclical, stateful agents.

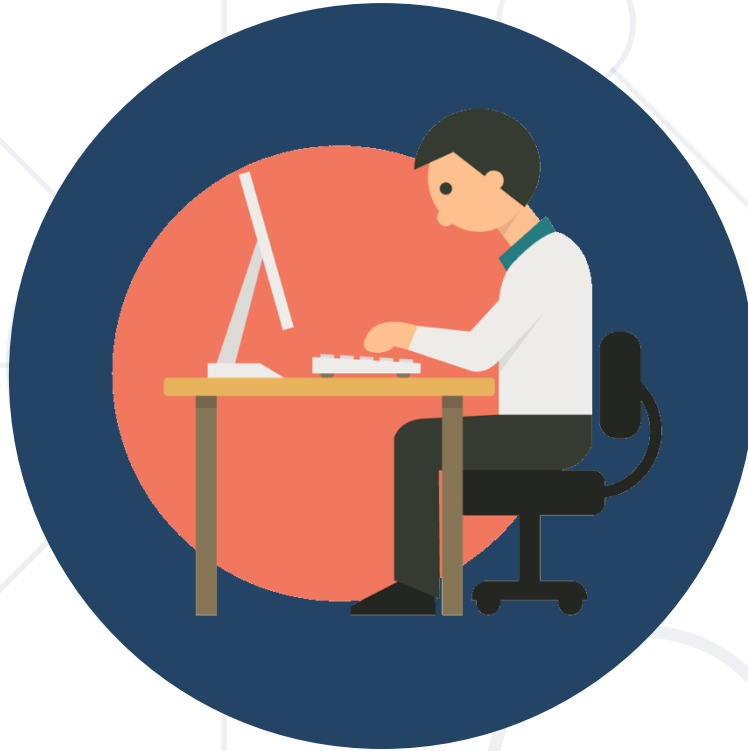
Chains vs. Graphs (Why LangGraph?)

- **The Limitation of Chains:** DAGs (Directed Acyclic Graphs) run step-by-step but cannot easily loop.
- **The Agentic Need:** True agents require cycles: Think → Act → Observe → Repeat.
- **The Solution:** LangGraph brings cyclical execution and robust state management.

- **Nodes:** Python functions or LCEL runnables (The "Workers").
- **Edges:** Define the flow between Nodes.
- **Conditional Edges:** Dynamic routing based on LLM decisions (The "Brain").

- **The StateGraph:** A global object (e.g., TypedDict or Pydantic) passed to every Node.
- **Message Passing:** Nodes don't overwrite the state; they return updates (append messages).
- **Shared Context:** All nodes operate on a single source of truth.

- **Tool Nodes:** Specific nodes dedicated to executing tools.
- **Dynamic Routing:** "If LLM calls a tool → Go to Tool Node. If LLM gives final answer → Go to END."
- **Separation of Concerns:** LLM decides, Tool executes.



Practice

Exercise – Build your first Agent with LangGraph



Reliability, Safety & Coordination

Making your agents production-ready and controllable.

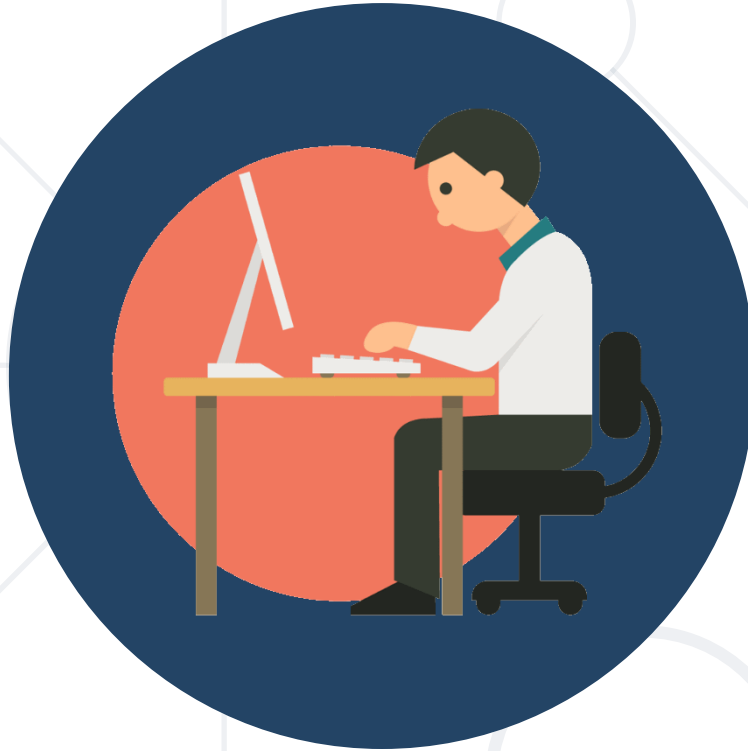
Persistence & Checkpointers (Memory)

- **Thread-Level Memory:** Save graph state to a database (SQLite, Postgres) automatically.
- **Cross-Session Memory:** Agents remember users across different conversations.
- **Resilience:** Recover instantly from server crashes.

- **State History**: Every step of the graph is versioned.
- **Time Travel**: "Rewind" the agent's state to a previous node.
- **Conflict Resolution**: Manually edit the state (e.g., fix a bad LLM thought) and fork the execution.

- **Parallel Execution:** Run multiple nodes simultaneously from a single point.
- **Fan-out / Fan-in:** Distribute sub-tasks to multiple workers and wait for all to finish before proceeding.
- **Use Case:** Speed up multi-document research.

- **Network Failures:** APIs and LLMs fail.
- **Built-in Reliability:** Attach retry policies directly to nodes (e.g., Retry-After).
- **Timeouts:** Prevent endless loops or stuck API calls.



Practice

Exercise – Enhance your first Agent with LangGraph



Multi-Agent Patterns

Scaling to complex, multi-step workflows.

Why Multi-Agent?

- Single agents struggle with **context limits** and **complex instructions**.
- The Solution: Divide and conquer.
- **Specialized agents** with specific prompts, tools, and roles.

Pattern 1 - The Network (Peer-to-Peer)

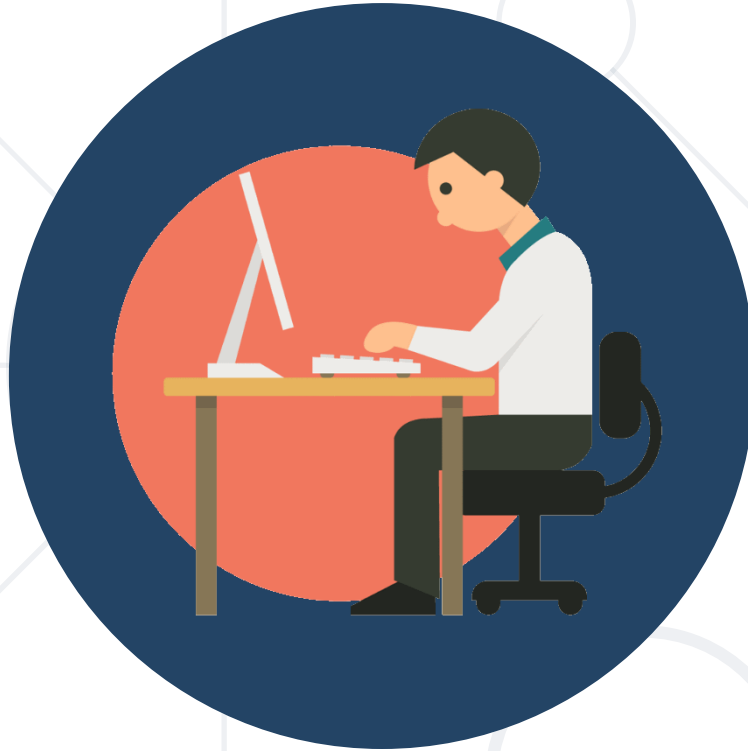
- Agents pass control directly to each other.
- Example: Researcher gathers data → Passes state to Writer → Writer drafts article.
- Good for linear but complex multi-step processes.

Pattern 2 - The Supervisor (Planners & Workers)

- Supervisor (Planner): An LLM that understands the overall goal but doesn't do the work.
- Workers: Specialized agents (e.g., Coder, Web Searcher).
- Flow: Supervisor delegates → Worker executes → Supervisor reviews and delegates next step.

Pattern 3 - Evaluator-Optimizer

- Generator Node: Creates a draft or solution.
- Evaluator Node: Critiques the output against specific criteria.
- Cycle: Loops between Generator and Evaluator until the score is acceptable.



Practice

Exercise – Solve a simple use case with multiple agents

- From **Chains** to **Graphs**: LangGraph moves beyond linear sequences to support cycles, allowing agents to iterate, self-correct, and reason deeply.
- **State** is King: Applications are built around a shared, persistent state, ensuring every node has the **context** it needs to make informed decisions.
- Divide and Conquer: **Multi-agent patterns** (Supervisors, Planners, Evaluators) allow you to scale complexity by orchestrating specialized mini-agents.
- The Power of **Control**: You no longer just "prompt and pray" — you design the **flow**, the guardrails, and the memory.

